

信息对抗技术实验报告

实验 7: Wireshark 网络抓包与协议分析合规实验

学院:	密码与网络空间安全学院
专业:	信息安全
学号:	2313177
姓名:	张昌源
实验日期:	2026 年 5 月

目录

1	实验目的	2
2	实验环境	2
3	实验原理	3
3.1	Wireshark 抓包与显示过滤器	3
3.2	DNS 域名解析	3
3.3	HTTP 明文请求与响应	4
3.4	TCP 三次握手与连接释放	4
3.5	HTTPS 与 TLS 加密	4
4	实验过程与结果分析	4
4.1	选择网卡并开始抓包	4
4.2	实验一: DNS 域名解析抓包分析	5
4.2.1	实验步骤	5
4.2.2	抓包结果	5
4.2.3	结果分析	6
4.3	实验二: UDP 数据包过滤分析	7
4.3.1	实验步骤	7
4.3.2	抓包结果与分析	7
4.4	实验三: 本地 HTTP 明文请求与响应分析	7
4.4.1	启动本地 Web 服务	7
4.4.2	HTTP 抓包结果	8
4.4.3	HTTP 明文传输风险分析	9
4.5	实验四: TCP 三次握手分析	9
4.5.1	筛选握手报文	9
4.5.2	三次握手过程	10
4.5.3	结果分析	11
4.6	实验五: TCP 连接释放过程分析	11
4.7	实验六: HTTPS/TLS 加密流量对比	13
4.7.1	实验步骤	13

4.7.2	抓包结果	14
4.7.3	HTTP 与 HTTPS 对比分析	14
5	报文封装层次分析	15
5.1	DNS 响应报文封装分析	15
5.2	TCP SYN 报文封装分析	15
5.3	HTTP GET 请求报文封装分析	15
6	实验总结	16

1 实验目的

本次实验围绕 Wireshark 网络抓包与基础协议分析展开。实验全程仅捕获本人设备、本地 Web 服务以及本人主动访问测试网站所产生的网络流量，不对公共网络中其他用户设备进行嗅探或分析。实验目标如下：

- 1. 掌握 Wireshark 的网卡选择、开始抓包、停止抓包以及保存抓包结果等基本操作。
- 2. 学会使用显示过滤器筛选 HTTP、DNS、TCP、UDP、TLS 等不同类型的数据包。
- 3. 通过 DNS 查询和响应数据包，理解域名解析过程以及 DNS 与 UDP 的关系。
- 4. 通过本地 HTTP 服务，观察明文 HTTP 请求、响应头和响应正文内容。
- 5. 通过 TCP 数据包，分析三次握手和连接释放过程，理解 TCP 可靠连接建立与终止机制。
- 6. 对比 HTTP 与 HTTPS 抓包结果，理解 TLS 加密对网络数据内容的保护作用。

2 实验环境

本次实验在 Windows 系统中进行，使用 Wireshark 对本机网络流量进行抓包。为保证 HTTP 明文交互过程可控，本实验同时搭建了一个本地 Web 服务，避免使用不明外部网站或采集他人流量。

表 1: 实验环境

项目	内容
操作系统	Windows 10 / Windows 11
抓包工具	Wireshark 4.6.5
浏览器	Microsoft Edge / Chrome 等主流浏览器
网络接口	WLAN、本地回环接口
本地 Web 服务	Python 简易 HTTP 服务器
本地访问地址	http://127.0.0.1:9000/
DNS 测试域名	www.baidu.com
HTTPS 测试地址	https://www.baidu.com
合规范围	仅抓取本机主动产生的网络流量，不采集他人设备数据

3 实验原理

3.1 Wireshark 抓包与显示过滤器

Wireshark 是一款常用的网络协议分析工具，可以对网卡经过的数据包进行捕获，并按照协议层次解析报文内容。在实际抓包时，网卡会持续收到操作系统、浏览器、局域网广播、DNS、TLS 等多种类型的数据包，因此不能依靠人工在完整列表中逐条查找目标报文，而应使用显示过滤器过滤出需要分析的协议或字段。

本次实验中使用的主要显示过滤器如表 2 所示。

表 2: 常用显示过滤器	
过滤表达式	作用说明
dns	仅显示 DNS 域名解析相关数据包。
dns.qry.name contains "baidu"	仅显示查询名称中包含 baidu 的 DNS 数据包。
udp.port == 53	观察与百度域名解析相关的 UDP 53 端口数据包。
&& dns.qry.name contains "baidu"	
http	仅显示 HTTP 协议数据包。
tcp.port == 9000 && http	显示与本地 9000 端口 Web 服务相关的 HTTP 报文。
tcp.port == 9000 && tcp.flags.syn == 1	筛选与本地 Web 服务连接建立相关的 SYN 报文。
tls	显示 HTTPS 访问过程中产生的 TLS 报文。

3.2 DNS 域名解析

用户访问网站时通常输入域名，而计算机实际通信需要使用 IP 地址。因此，访问网站前通常需要先进行 DNS 解析。客户端向 DNS 服务器发送查询请求，请求某个域名对应的 A 记录或 AAAA 记录；DNS 服务器返回解析结果后，客户端再依据 IP 地址发起后续连接。

DNS 查询通常使用 UDP 作为传输层协议，常见服务器端口为 53。由于 UDP 不需要建立连接，适合短小、快速的查询型通信，因此普通域名解析通常可以在 Wireshark 中通过 dns 或 udp.port == 53 过滤到。

3.3 HTTP 明文请求与响应

HTTP 是应用层请求-响应协议。浏览器作为客户端向服务器发出请求，服务器返回状态码、响应头和响应体。对于未加密的 HTTP，Wireshark 可以直接显示请求行、请求头、响应头以及 HTML 正文内容。本实验使用本地 Python HTTP 服务器作为测试对象，可以稳定观察到明文 HTTP 交互过程。

典型 HTTP 请求与响应如下所示：

```
GET / HTTP/1.1
Host: 127.0.0.1:9000
User-Agent: Mozilla/5.0 ...

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.2
Content-type: text/html
```

3.4 TCP 三次握手与连接释放

HTTP 通常运行在 TCP 之上。客户端发送 HTTP 请求前，需要先通过三次握手建立可靠连接：客户端发送 SYN；服务器返回 SYN+ACK；客户端再返回 ACK，连接建立完成。数据传输结束后，通信双方通过 FIN 与 ACK 报文逐步释放连接。

Wireshark 中可以根据 TCP 标志位观察上述过程。SYN 标志表示请求建立连接，ACK 表示确认，FIN 表示一方准备关闭连接。

3.5 HTTPS 与 TLS 加密

HTTPS 在 HTTP 与 TCP 之间加入 TLS 安全层。TLS 可以为通信双方提供完整性、身份认证和机密性保护。常规抓包时，Wireshark 能够看到 TLS 握手、SNI、证书协商以及加密后的 Application Data，但不能像明文 HTTP 那样直接看到 GET 请求、Cookie 或 HTML 正文。

4 实验过程与结果分析

4.1 选择网卡并开始抓包

打开 Wireshark 后，在网卡列表中选择当前正在使用的网络接口。本机通过 WLAN 联网，因此首先选择 WLAN 接口进行抓包。进入抓包界面后，列表中会实时出现系统

后台通信、局域网广播、DNS、TLS 等多种数据包，这是正常现象。实验分析时主要依靠显示过滤器筛选目标流量。

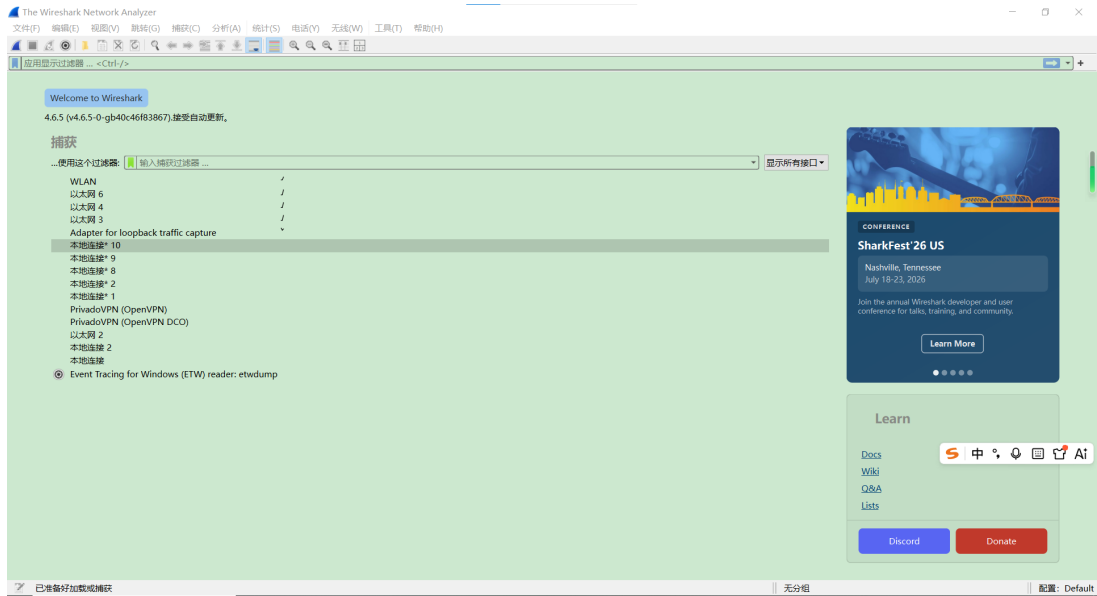


图 1: Wireshark 主界面与 WLAN 网卡选择

4.2 实验一：DNS 域名解析抓包分析

4.2.1 实验步骤

为了避免浏览器缓存、后台请求和局域网广播造成干扰，本实验使用命令行明确触发一次 DNS 查询。具体步骤如下：

- 1. 在 Wireshark 中选择 WLAN 接口并开始抓包。
- 2. 在过滤栏输入: `dns.qry.name contains "baidu"`。
- 3. 打开命令提示符，先执行 `ipconfig /flushdns` 清空本地 DNS 缓存。
- 4. 继续执行 `nslookup www.baidu.com`, 主动产生对 `www.baidu.com` 的 DNS 查询。
- 5. 回到 Wireshark，观察筛选后的 DNS Query 与 DNS Response。

4.2.2 抓包结果

图 2 为过滤后的 DNS 抓包结果。可以看到，客户端发起了对 `www.baidu.com` 的 A 记录和 AAAA 记录查询，随后 DNS 服务器返回了对应的解析结果。

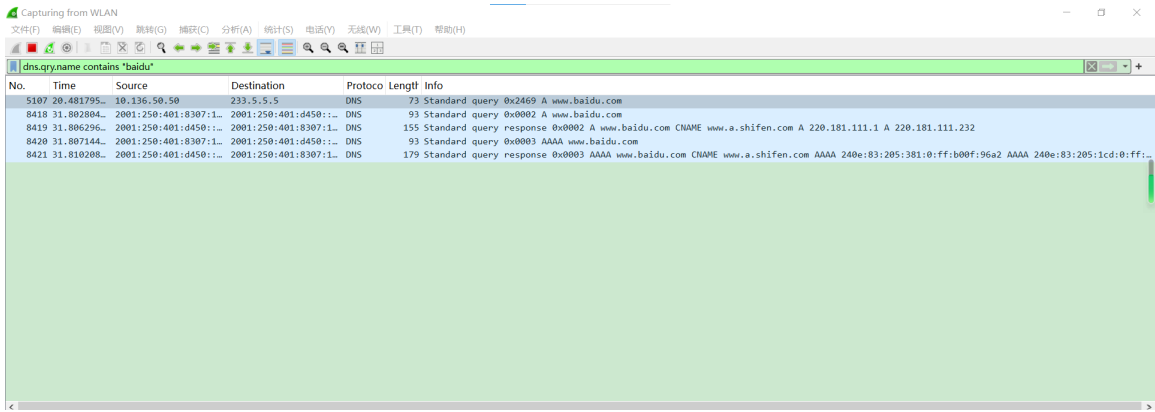


图 2: 使用 dns.qry.name contains "baidu" 过滤百度域名解析数据包

进一步展开 DNS Response 的 Answers 字段, 可以看到 **www.baidu.com** 首先被解析为 CNAME 记录 **www.a.shifen.com**, 然后返回对应的 IPv4 地址 **220.181.111.1** 和 **220.181.111.232**。

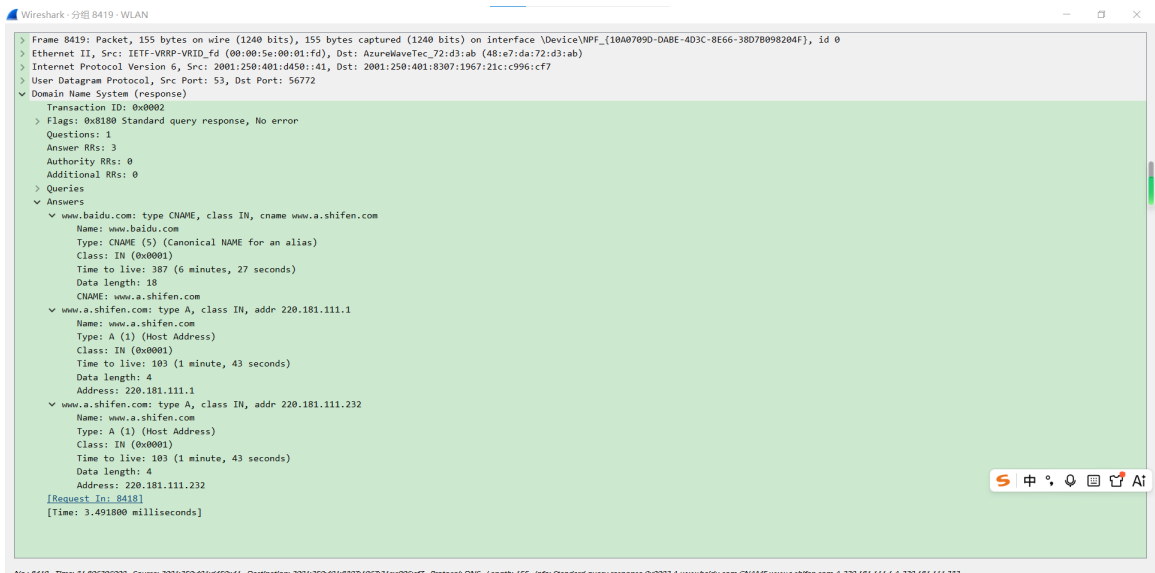


图 3: DNS Response 中 Answers 字段展开结果

4.2.3 结果分析

从抓包结果可以看出, 访问网站前, 客户端需要先通过 DNS 查询获得目标服务器 IP 地址。DNS 响应中包含 CNAME 和 A 记录, 说明域名解析可能不是一步得到最终 IP, 而是先经过别名记录, 再返回实际主机地址。该过程体现了域名系统将易记域名映射到网络层 IP 地址的作用。

4.3 实验二：UDP 数据包过滤分析

4.3.1 实验步骤

在完成 DNS 抓包后，不重新抓包，直接将 Wireshark 显示过滤器改为：

```
udp.port == 53 && dns.qry.name contains "baidu"
```

该过滤器用于筛选与百度域名解析相关、且使用 UDP 53 端口的数据包。

4.3.2 抓包结果与分析

图 4 显示，在 UDP 53 端口过滤条件下，仍然可以看到 `www.baidu.com` 的 DNS Query 与 Response，说明本次域名解析数据包在传输层使用 UDP 承载。

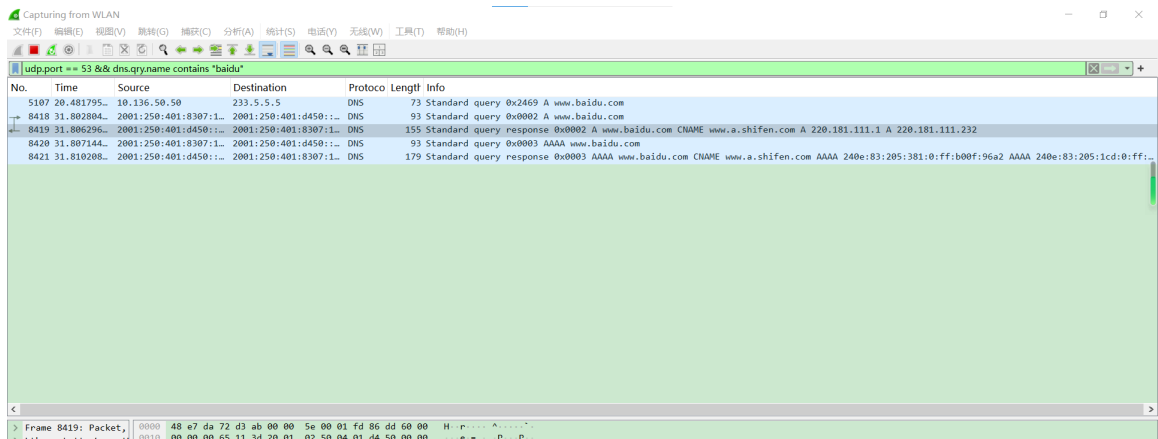


图 4: 使用 `udp.port == 53` 观察 DNS 所使用的 UDP 数据包

从传输层角度看，DNS 查询数据量较小，对可靠连接要求不高，因此使用 UDP 可以减少连接建立开销，提高查询效率。本次实验中 DNS Response 的源端口为 53，目的端口为客户端临时端口，符合典型 DNS 解析通信特征。

4.4 实验三：本地 HTTP 明文请求与响应分析

4.4.1 启动本地 Web 服务

为了稳定观察明文 HTTP 报文，本实验在本机目录下准备一个简单网页，并通过 Python 简易 HTTP 服务器监听 9000 端口。启动命令如下：

```
python -m http.server 9000
```

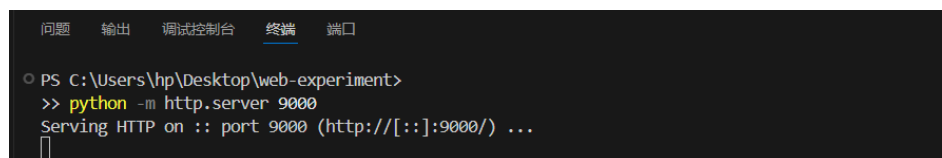


图 5: 启动本地 Python HTTP 服务

随后在浏览器中访问:

`http://127.0.0.1:9000/`

页面正常显示, 说明本地 HTTP 服务已经启动并可以被浏览器访问。



图 6: 浏览器访问本地 HTTP 测试页面

4.4.2 HTTP 抓包结果

在 Wireshark 中使用 `tcp.port == 9000 && http` 或 `http` 过滤器, 可以筛选出本地浏览器与本地 Web 服务之间的 HTTP 请求与响应。为了观察完整应用层内容, 选中 HTTP 数据包后右键选择“追踪 TCP 流”。结果如图 7 所示。

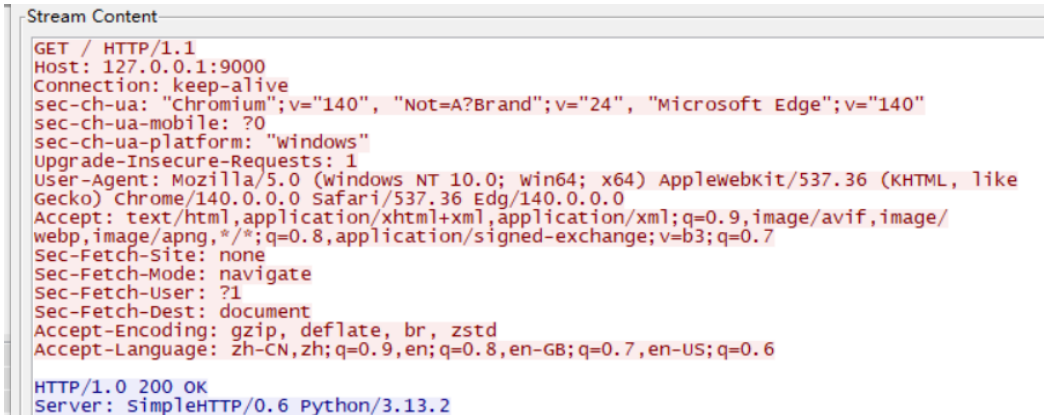


图 7: 追踪 TCP 流查看明文 HTTP 请求与响应

可以看到，浏览器向服务器发送了如下请求：

```
GET / HTTP/1.1
Host: 127.0.0.1:9000
Connection: keep-alive
User-Agent: Mozilla/5.0 ...
Accept: text/html,application/xhtml+xml,...
```

服务器返回：

```
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.2
Content-type: text/html
```

这说明 HTTP 明文传输时，请求方法、访问路径、主机地址、浏览器标识以及响应状态码等信息都可以直接在抓包工具中观察到。

4.4.3 HTTP 明文传输风险分析

明文 HTTP 的特点是可读性强，便于实验分析，但安全性较弱。如果网站使用 HTTP 传输敏感信息，攻击者在具备抓包条件时可能观察到请求头、表单字段或 Cookie 等信息。因此，真实网络服务通常应使用 HTTPS，通过 TLS 对 HTTP 内容进行加密保护。

4.5 实验四：TCP 三次握手分析

4.5.1 筛选握手报文

在本地 HTTP 实验中，浏览器访问 127.0.0.1:9000 前会先与服务器建立 TCP 连接。使用以下过滤器可以筛选与 9000 端口相关的 SYN 报文：

```
tcp.port == 9000 && tcp.flags.syn == 1
```

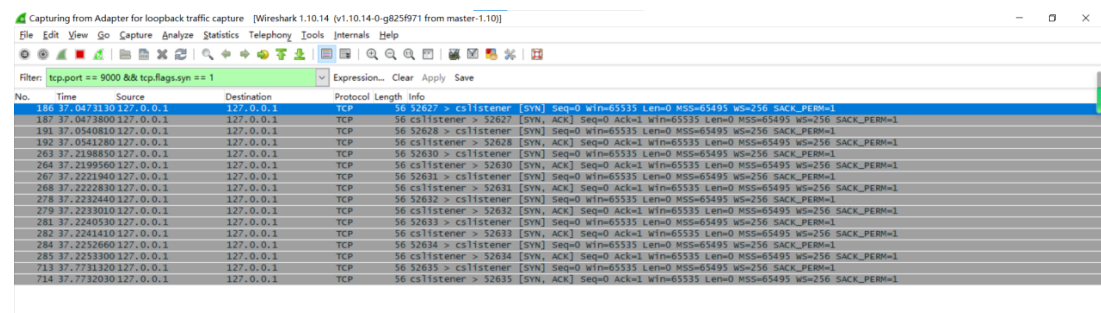


图 8: 使用 TCP SYN 标志位筛选连接建立阶段报文

4.5.2 三次握手过程

选取同一 TCP 连接中的三个关键报文进行分析：

- 1. 第一次握手：客户端临时端口 52627 向服务器 9000 端口发送 SYN，请求建立连接。
- 2. 第二次握手：服务器 9000 端口返回 SYN+ACK，表示同意建立连接并确认客户端请求。
- 3. 第三次握手：客户端再次发送 ACK，确认服务器响应，TCP 连接建立完成。

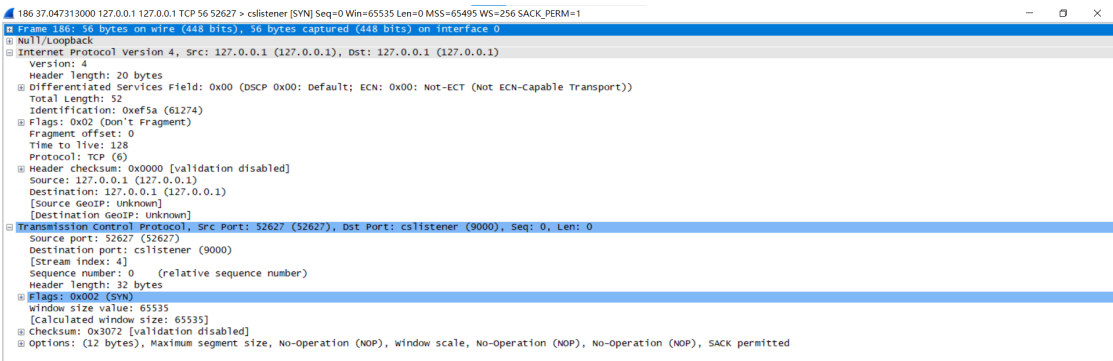


图 9: 第一次握手：客户端发送 SYN 报文

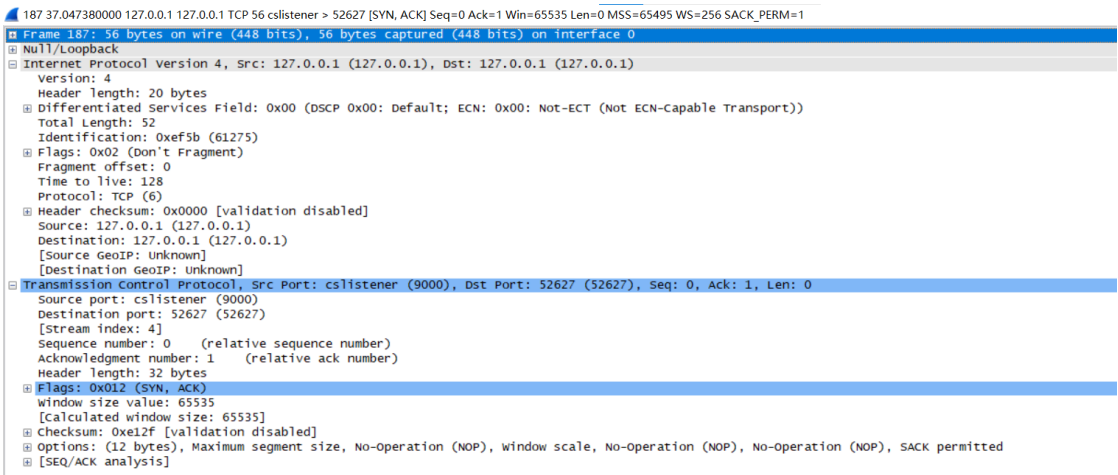


图 10: 第二次握手: 服务器返回 SYN+ACK 报文

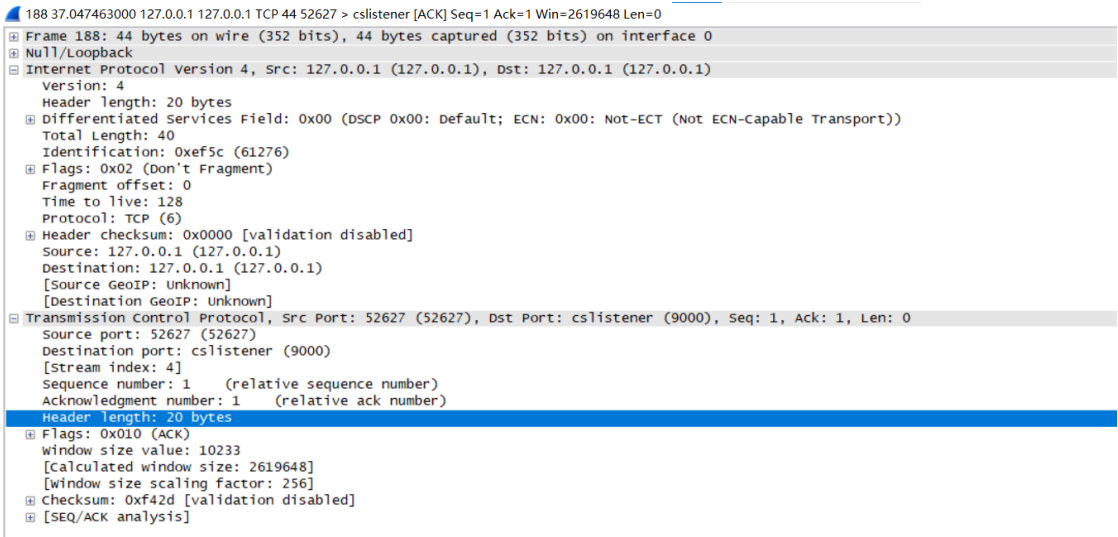


图 11: 第三次握手: 客户端发送 ACK 报文

4.5.3 结果分析

三次握手完成后，双方已经确认彼此的收发能力，并完成初始序列号同步。随后浏览器才会在该 TCP 连接之上发送 HTTP GET 请求。由此可见，HTTP 请求虽然属于应用层内容，但它依赖 TCP 在传输层提供可靠连接。

4.6 实验五: TCP 连接释放过程分析

在 HTTP 数据传输结束后，通信双方通过 FIN 与 ACK 报文释放 TCP 连接。本实验中观察到的关闭过程如下：

表 3: TCP 连接释放过程

阶段	端口方向	含义
FIN+ACK	9000 → 52627	服务器端表示数据发送完毕，准备关闭连接。
ACK	52627 → 9000	客户端确认收到服务器关闭请求。
FIN+ACK	52627 → 9000	客户端也表示准备关闭连接。
ACK	9000 → 52627	服务器确认客户端关闭请求，连接最终释放。

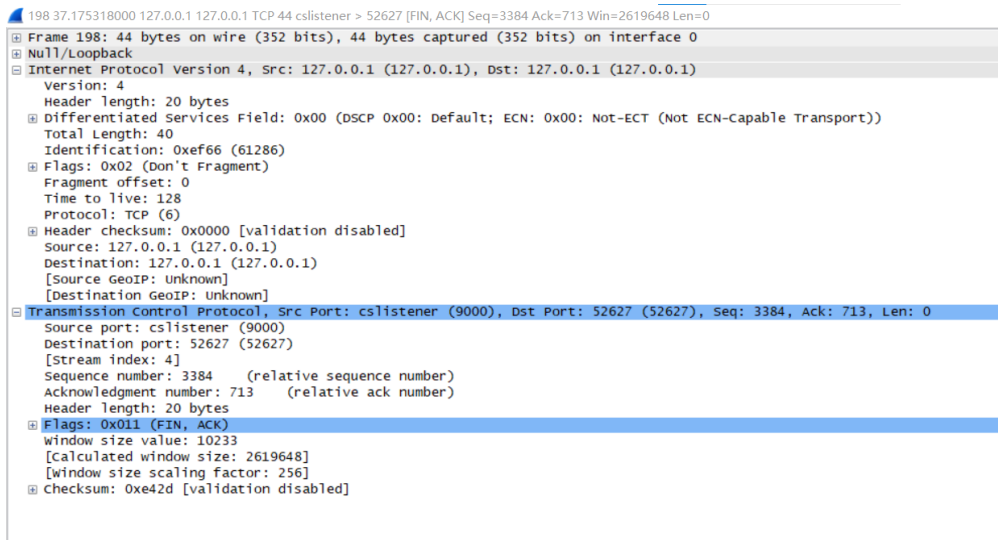


图 12: 服务器端发送 FIN+ACK 报文

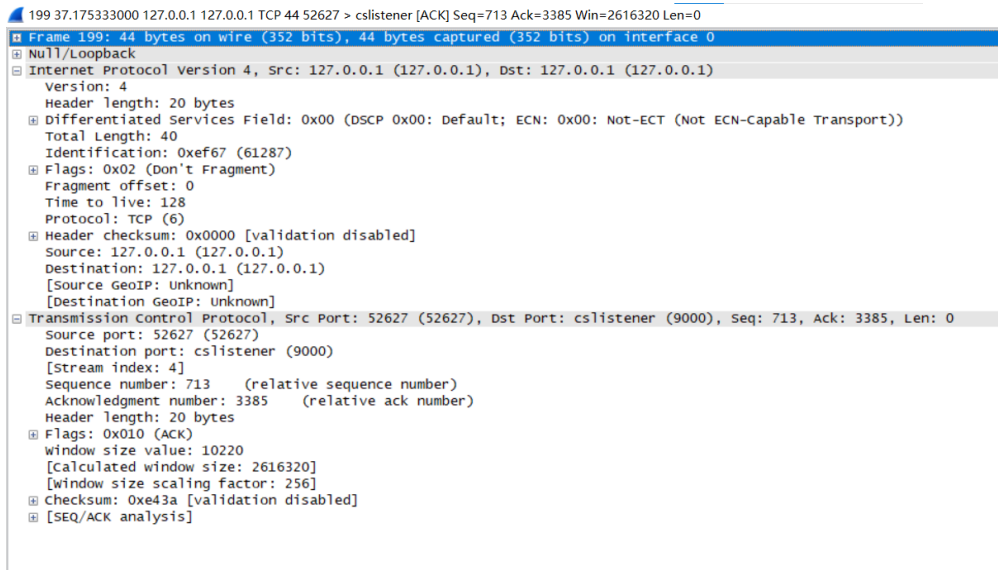


图 13: 客户端确认服务器关闭请求

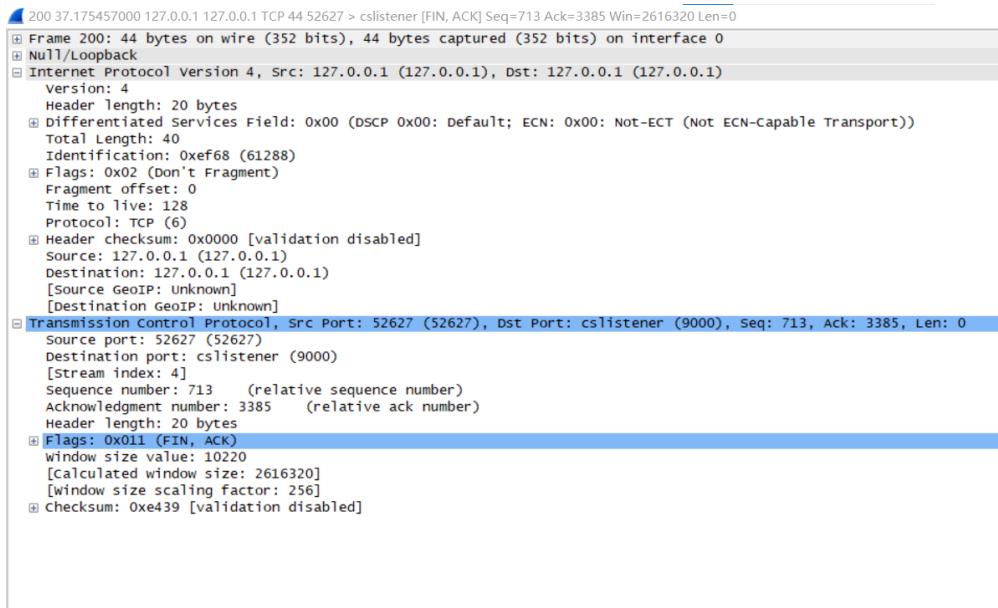


图 14: 客户端发送 FIN+ACK 报文

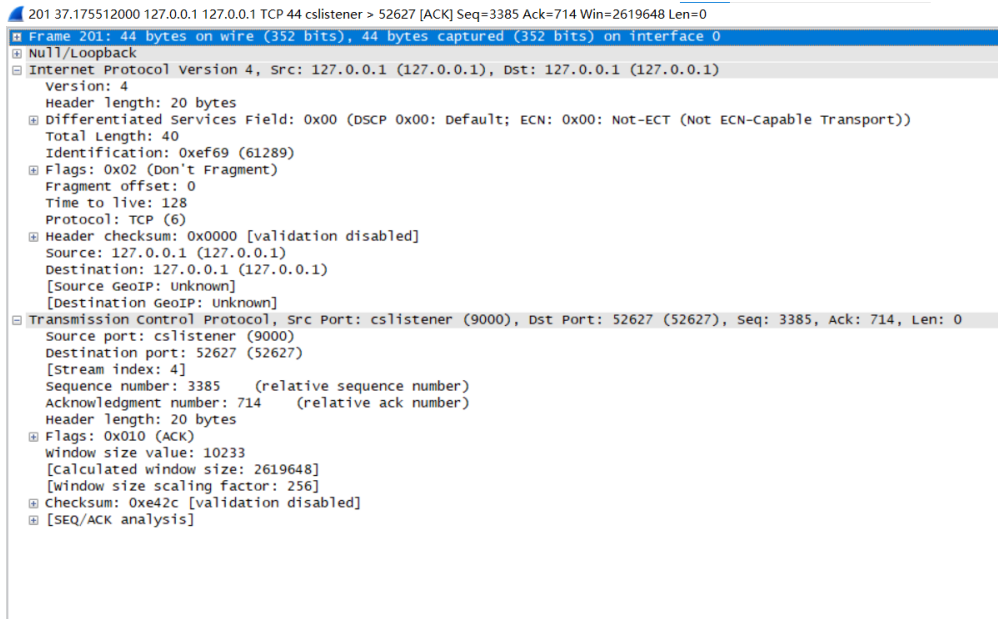


图 15: 服务器端返回 ACK，连接释放完成

连接释放过程说明 TCP 不仅负责建立可靠连接，也负责在通信结束后有序释放连接资源，避免双方状态不一致。

4.7 实验六: HTTPS/TLS 加密流量对比

4.7.1 实验步骤

为了与 HTTP 明文结果进行对比，重新开始抓包，在 Wireshark 过滤栏中输入：

tls

随后在浏览器中访问：

https://www.baidu.com

4.7.2 抓包结果

图 16 为 TLS 抓包结果。可以看到，访问 HTTPS 网站时，Wireshark 中主要显示 TLS 握手报文和加密后的应用数据，例如 Client Hello、Server Hello、Change Cipher Spec、Encrypted Handshake Message 和 Application Data 等。

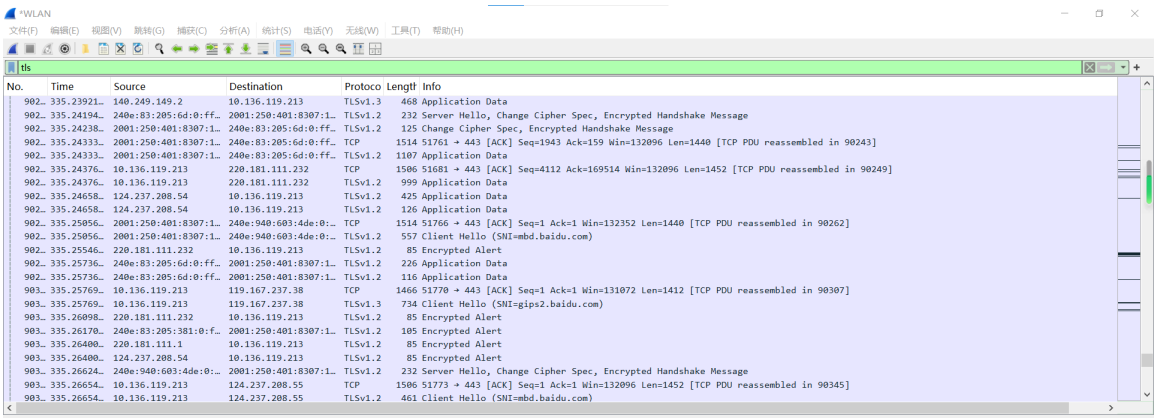


图 16: HTTPS 访问过程中的 TLS 报文

4.7.3 HTTP 与 HTTPS 对比分析

与本地 HTTP 抓包相比，HTTPS 抓包具有明显差异：在 HTTP 明文实验中，可以直接看到 GET / HTTP/1.1、Host、User-Agent 和响应头；而在 HTTPS 实验中，Wireshark 只能看到 TLS 握手与加密后的应用数据，不能直接看到 HTTP 请求正文、Cookie 或网页 HTML 内容。

这说明 HTTPS/TLS 能够有效保护应用层数据的机密性。即使攻击者能够捕获数据包，也难以直接读取其中的具体网页内容或敏感字段。本实验也进一步说明：当前主流网站采用 HTTPS 是网络安全的重要基础措施。

5 报文封装层次分析

5.1 DNS 响应报文封装分析

以 `www.baidu.com` 的 DNS Response 为例, 该报文在 Wireshark 中可以展开为多个层次:

- 数据链路层: Ethernet II, 包含源 MAC 地址和目的 MAC 地址, 用于局域网内帧转发。
- 网络层: IPv6, 包含源 IPv6 地址和目的 IPv6 地址, 用于网络层寻址与传输。
- 传输层: UDP, 源端口为 53, 目的端口为客户端临时端口, 说明该报文来自 DNS 服务器。
- 应用层: DNS, 包含查询编号、响应标志、问题数量、回答数量以及 Answers 字段。

在 Answers 字段中, 可以看到 `www.baidu.com` 对应的 CNAME 与 A 记录, 说明 DNS 响应不仅包含“是否解析成功”, 还包含具体的资源记录类型和返回地址。

5.2 TCP SYN 报文封装分析

以客户端发出的 SYN 报文为例, 其封装层次如下:

- 数据链路层: Null/Loopback 或 Ethernet II, 用于本机或局域网传输。
- 网络层: IPv4, 源地址和目的地址均为 `127.0.0.1`, 表示本地回环通信。
- 传输层: TCP, 源端口为客户端临时端口, 目的端口为 9000, Flags 字段为 SYN。
- 应用层: 无。SYN 报文仅用于建立连接, 不携带 HTTP 应用层数据。

该报文说明 TCP 连接建立阶段发生在应用层数据传输之前, 只有在三次握手成功后, HTTP 请求才会正式发送。

5.3 HTTP GET 请求报文封装分析

以 HTTP GET 请求为例, 其封装层次如下:

- 数据链路层: 完成本机或局域网内帧传输。
- 网络层: IPv4, 源地址和目的地址均为 `127.0.0.1`。
- 传输层: TCP, 源端口为客户端临时端口, 目的端口为 9000, Flags 通常包含 PSH、ACK, 表示将数据交付给上层应用。

- 应用层:HTTP,包含 GET / HTTP/1.1、Host: 127.0.0.1:9000、User-Agent、Accept 等字段。

该报文体现了应用层数据在网络传输中的逐层封装过程: HTTP 数据先交给 TCP, TCP 加上端口与序列号信息后交给 IP, IP 再封装到链路层帧中发送。

6 实验总结

通过本次实验,我对 Wireshark 抓包和常见网络协议的工作方式有了更直观的认识。首先,在 Wireshark 中直接开始抓包后会出现大量背景流量,这些数据并非异常,而是系统正常通信、局域网广播、浏览器后台请求等共同产生的结果。因此,实验分析不能依靠人工翻找数据包,而应使用显示过滤器精确定位目标协议或字段。

其次,通过 DNS 抓包可以看到域名解析的完整过程。客户端查询 `www.baidu.com`, DNS 服务器返回 CNAME 和 A 记录,使浏览器能够根据解析得到的 IP 地址继续访问目标服务器。通过 `udp.port == 53` 过滤结果,也验证了 DNS 查询通常使用 UDP 进行传输。

再次,通过本地 HTTP 服务实验,我观察到了明文 HTTP 请求和响应内容。Wireshark 可以直接显示 GET / HTTP/1.1、Host、User-Agent、HTTP/1.0 200 OK 等内容,这说明 HTTP 明文传输虽然便于分析,但也存在泄露信息的风险。

最后,通过 TLS 抓包对比可以看到,HTTPS 访问过程中只能观察到 TLS 握手和加密后的 Application Data,而无法直接读取网页正文或具体 HTTP 请求内容。这说明 HTTPS/TLS 对保护网络通信内容具有重要意义。

本实验全过程仅围绕本人设备、本地 Web 服务和本人主动访问的网站展开,未采集或分析他人网络流量,符合网络安全实验的合法合规要求。